Understanding Python Context Managers

Practical Patterns with Files and SQLite

RESOURCE LIFECYCLES

Every resource you open must reliably close

- A file handle is a real system resource. Leaving it open can leak memory or block other processes from accessing the file.
- The fragile pattern opens the file and calls .close() manually — this only works if no error happens first.
- The safe pattern wraps the same code in try/finally, so the file closes even if process() raises an error.
- The with statement expresses this exact guarantee in one clean line, with no boilerplate to repeat.

slide_02_fragile_vs_safe.py

```
# Fragile: close() is skipped if process() raises
f = open('data.txt', 'w')
try:
    process(f)
finally:
    f.close()

# Safe and idiomatic: cleanup is guaranteed
with open('data.txt', 'w') as f:
    process(f)
```

DESIGN INSIGHT: Key Principle

A with block guarantees f.close() runs on the way out, whether process(f) succeeds, fails, or returns early.

LANGUAGE SEMANTICS

The with statement calls __enter__ and __exit__ for you

- When the with block starts, Python calls the object's __enter__() method — for a file, this returns the file object itself.
- That returned object is then assigned to the variable named after as — here, f.
- Code inside the block runs normally, using f exactly as you would use any file object.
- When the block ends, Python automatically calls __exit__(), which closes the file whether or not an exception occurred.

slide_03_with_statement_translation.py

```
# What you write:
with open('data.txt', 'w') as f:
    f.write('hello')

# What Python actually does:
f_ctx = open('data.txt', 'w')
f = f_ctx.__enter__()
try:
    f.write('hello')
except BaseException as exc:
    if not f_ctx.__exit__(type(exc), exc,
                          exc.__traceback__):
        raise
else:
    f_ctx.__exit__(None, None, None)
```

DESIGN INSIGHT: Real Translation

A file object already implements __enter__ and __exit__, so open() can be used directly after with.

CORE PROTOCOL

Two dunder methods define the entire context lifecycle

- __enter__(self) runs when the with block starts. It opens the file and returns the object that as will bind to.
- __exit__(self, exc_type, exc_val, tb) runs when the with block ends, whether it finished normally or raised an exception.
- Returning False from __exit__ allows any exception raised inside the block to keep propagating as normal.
- Returning True would silently suppress that exception instead — almost never what you actually want.

slide_04_filemanager_class.py

```
class FileManager:
    def __init__(self, path, mode):
        self.path = path
        self.mode = mode

    def __enter__(self):
        self.file = open(self.path, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, tb):
        self.file.close()
        return False  # propagate exceptions

# Usage
with FileManager('notes.txt', 'w') as f:
    f.write('logged safely\n')
```

DESIGN INSIGHT: Under the Hood

FileManager is a plain class — there is no magic beyond Python calling __enter__ and __exit__ at the right times.

STANDARD LIBRARY

@contextmanager turns a generator into a context manager

- A generator function can act as a full context manager, without writing a class with two dunder methods.
- Everything before yield is the setup phase — here, that means opening the file.
- yield pauses the generator and hands the file object to the with block to use.
- Everything after yield is the teardown phase, wrapped in finally so it always runs, even after an error.

slide_05_contextmanager_decorator.py

```
from contextlib import contextmanager

@contextmanager
def managed_file(path, mode):
    f = open(path, mode)
    try:
        yield f
    finally:
        f.close()  # guaranteed cleanup

# Usage
with managed_file('log.txt', 'w') as f:
    f.write('entry\n')
```

DESIGN INSIGHT: Why finally Matters

Without try/finally, an exception inside the with block would skip f.close() and leak the open file handle.

DATA INTEGRITY

A transaction must commit on success or roll back on failure

- This mirrors a normal SQL session: open a connection, run the queries, commit or roll back, then close.
- conn.commit() only runs if the whole with block finishes without raising an exception.
- conn.rollback() undoes any partial changes if an exception occurs partway through the transaction.
- The bare raise re-raises the original exception after rolling back, so the caller still sees the failure.

slide_06_db_transaction.py

```python
import sqlite3
from contextlib import contextmanager

@contextmanager
def db_transaction(path):
    conn = sqlite3.connect(path)
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()  # no connection leaks

# Usage
with db_transaction('shop.db') as conn:
    conn.execute(
        'INSERT INTO orders VALUES (?)', (42,))
```

DESIGN INSIGHT: Commit Placement

conn.commit() sits inside the try, right after yield, so it only runs when the caller's block completes cleanly.

PERFORMANCE PROFILING

A class-based manager can track state across a query

- A class-based context manager is useful when you need to keep and use information after the block exits.
- __enter__ opens a cursor on an existing connection and returns it, bound to the as variable.
- __exit__ measures how long the query took, closes the cursor, and still lets any query error propagate.
- This same shape works for query counters, connection pools, or audit logging.

slide_07_query_timer.py

```
class QueryTimer:
    def __init__(self, conn):
        self.conn = conn

    def __enter__(self):
        self.start = time.perf_counter()
        self.cursor = self.conn.cursor()
        return self.cursor

    def __exit__(self, exc_type, exc_val, tb):
        elapsed = time.perf_counter() - self.start
        print(f'Query took {elapsed:.4f}s')
        self.cursor.close()
        return False

# Usage
with QueryTimer(conn) as cur:
    cur.execute('SELECT * FROM orders')
    rows = cur.fetchall()
```

DESIGN INSIGHT: Reusable Profiling

Because the connection is passed in rather than created inside, the same QueryTimer wraps any query, anywhere.

RESOURCE POOLING

ExitStack manages a variable number of resources

- When the number of files to open is only known at runtime, you can't nest a fixed number of with blocks.
- stack.enter_context() opens each file and registers it with one shared ExitStack instance.
- When the block exits, every registered file is closed automatically, in reverse order.
- If opening the third file fails, the first two are still closed safely — nothing is left open.

slide_08_exitstack.py

```
from contextlib import ExitStack

filenames = ['a.txt', 'b.txt', 'c.txt']

with ExitStack() as stack:
    handles = [
        stack.enter_context(open(name, 'w'))
        for name in filenames
    ]
    for handle in handles:
        handle.write('managed safely\n')

# All three files are closed here,
# even if one of the writes had failed.
```

DESIGN INSIGHT: Popular Uses

Batch file writers, multi-database routers, and network socket pools all rely on this dynamic registration pattern.

VERIFICATION

Test both the success and the failure path

- The happy-path test runs the transaction normally and confirms the row was actually committed.
- The failure-path test raises inside the block and confirms nothing was written to the table.
- An in-memory SQLite database (':memory:') is fast enough to use directly in tests, with no mocking required.
- Testing both paths together catches a context manager that quietly swallows errors or commits partial work.

slide_09_test_rollback.py

```
@contextmanager
def db_transaction(conn):
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise

def test_rollback_on_error():
    conn = sqlite3.connect(':memory:')
    conn.execute('CREATE TABLE t (id INTEGER)')
    try:
        with db_transaction(conn):
            conn.execute('INSERT INTO t VALUES (1)')
            raise ValueError('simulated failure')
    except ValueError:
        pass
    count = conn.execute(
        'SELECT COUNT(*) FROM t').fetchone()[0]
    assert count == 0  # rollback verified
```

DESIGN INSIGHT: No Mocking Needed

Because SQLite can run entirely in memory, the test stays fast while still exercising the real commit and rollback code.

DESIGN PATTERNS

Avoid two common context manager mistakes

- Before writing one, ask: what resource is being managed, what must always run afterward, and should errors propagate or be caught?
- Mistake 1: opening the file inside __init__ leaves no clear boundary, and risks a leak if setup fails before __enter__ even runs.
- Mistake 2: accidentally returning True from __exit__, which silently swallows every exception raised inside the block.
- The fix: acquire the resource inside __enter__, and only return True from __exit__ when you deliberately want to suppress an error.

slide_10_anti_patterns.py

```python
# Risky: file opened before __enter__ even runs
class RiskyFile:
    def __init__(self, path):
        self.f = open(path, 'w')

    def __enter__(self):
        return self.f

    def __exit__(self, *exc):
        self.f.close()

# Safer: acquisition tied to the with block
class SaferFile:
    def __init__(self, path):
        self.path = path

    def __enter__(self):
        self.f = open(self.path, 'w')
        return self.f

    def __exit__(self, *exc):
        self.f.close()
        return False  # never swallow errors
```

DESIGN INSIGHT: Tie Setup to __enter__

If constructing RiskyFile raises after the file is opened, the with block never starts — so __exit__ never runs to close it.

REAL-WORLD USE

Use a context manager wherever a resource must be released

- Why: programs that run for a long time, such as web servers, data pipelines and hospital or banking systems, fail when leaked file handles, connections or locks build up after errors.
- Why: a with block makes clean-up automatic on success, on an exception and on an early return, so no code path can skip it.
- Where: files and network sockets, database connections and transactions, thread locks, and temporary files or directories.
- Where: in tests, unittest.mock.patch and pytest.raises use with blocks to restore or check state.

slide_11_real_world_uses.py

```python
import sqlite3, threading

# Files: always closed
with open('report.csv', 'w') as f:
    f.write('id,total\n')

# Locks: always released
lock = threading.Lock()
with lock:
    print('updating shared data')

# Database: commit, or roll back on error
conn = sqlite3.connect('shop.db')
with conn:
    conn.execute('CREATE TABLE IF NOT EXISTS t (x)')
conn.close()  # the with block does not close it
```

DESIGN INSIGHT: Rule of Thumb

If code opens, connects to, acquires or creates something that must later be closed, released, committed or removed, put that work inside a with block.